My Name: Siddharth Puhan

Task: Week 3 – Technical Learning & Implementation

Week 3 – Technical Learning & Implementation Report

> *Note: code and outputs in this report are small examples written to explain each topic. They are marked "illustrative" where they are not from a recorded run.*

1. Flask API Introduction

What is Flask?

Flask is a small Python web framework. It lets us build web pages and APIs with very little code. It gives us routing and request handling, and we add other tools only when needed.

Basic Flask application and the /hello route

```
# app.py
from flask import Flask, jsonify

app = Flask(__name__)

@app.route("/hello")
def hello():
    return jsonify({"message": "Hello from Flask"})

if __name__ == "__main__":
    app.run(debug=True)
```

- Install: `pip install flask`. Run: `python app.py`.
- The server starts on `http://127.0.0.1:5000`.
- `@app.route("/hello")` links the URL `/hello` to the function below it.

Returning JSON

`jsonify()` turns a Python dictionary into JSON and sets the header `Content-Type: application/json`. Browsers and JavaScript can then read it easily.

```
# Illustrative request and response
GET http://127.0.0.1:5000/hello

200 OK
{"message": "Hello from Flask"}
```

Flask vs FastAPI

Point	Flask	FastAPI
Style	Simple and minimal	Modern, uses Python type hints
Validation	Manual or with extra libraries	Built in (Pydantic)
Docs	Not automatic	Automatic Swagger docs at /docs
Async	Limited	Built-in async support
Good for	Small apps, learning	Fast APIs, bigger projects

2. JavaScript and Flask Integration

How they communicate

The frontend (JavaScript in the browser) sends an HTTP request to the Flask backend. Flask runs the matching function and sends back a JSON response. JavaScript reads the JSON and updates the page.

```
Browser (JS)  --- HTTP request (GET /hello) --->  Flask
Browser (JS)  <-- JSON response --------------   Flask
```

Calling the API with fetch()

```
// GET request
fetch("http://127.0.0.1:5000/hello")
  .then(res => res.json())
  .then(data => {
    document.getElementById("output").textContent = data.message;
  })
  .catch(err => console.error("Error:", err));
```

A POST request sends data in the body. Flask reads it with `request.get_json()`.

```
// JavaScript
fetch("http://127.0.0.1:5000/greet", {
  method: "POST",
  headers: { "Content-Type": "application/json" },
  body: JSON.stringify({ name: "Siddharth" })
})
  .then(res => res.json())
  .then(data => console.log(data));

# Flask
from flask import request

@app.route("/greet", methods=["POST"])
def greet():
    data = request.get_json()
    return jsonify({"message": "Hello " + data["name"]})
```

Handling CORS

- CORS (Cross-Origin Resource Sharing) is a browser rule. A page on one origin (for example `localhost:3000`) cannot read a response from another origin (`localhost:5000`) unless the server allows it.
- If CORS is not set, the browser shows an error in the console like "blocked by CORS policy".
- Fix in Flask: install `flask-cors` and enable it.

```
# pip install flask-cors
from flask_cors import CORS

CORS(app)  # allows all origins (fine for learning)
# For real projects: CORS(app, origins=["http://localhost:4200"])
```

Basic request and response flow

- User clicks a button on the page.
- JavaScript calls `fetch()` with the API URL.
- Browser checks CORS. Flask receives the request and runs the route function.
- Flask returns JSON. JavaScript reads it and shows it on the page.

3. Angular Basics

What is Angular?

Angular is a frontend framework made by Google. It uses TypeScript and comes with many things built in, such as routing, forms, and an HTTP client. It is used to build single-page applications (SPAs).

Components

A component is a building block of the screen. Each one has a TypeScript class, an HTML template, and CSS styles.

```
// hello.component.ts
import { Component } from '@angular/core';

@Component({
  selector: 'app-hello',
  standalone: true,
  template: `<h2>{{ title }}</h2>`,
})
export class HelloComponent {
  title = 'Hello Angular';
}
```

We use it in another template with `<app-hello></app-hello>`. Components make the UI reusable and easy to manage.

Basic routing

Routing shows different components for different URLs without reloading the page.

```
// app.routes.ts
import { Routes } from '@angular/router';
import { HomeComponent } from './home/home.component';
import { AboutComponent } from './about/about.component';

export const routes: Routes = [
  { path: '', component: HomeComponent },
  { path: 'about', component: AboutComponent },
];

<!-- app.component.html -->
<a routerLink="/">Home</a>
<a routerLink="/about">About</a>
<router-outlet></router-outlet>
```

Angular application structure

```
my-app/
  src/
    app/
      app.component.ts      root component
      app.routes.ts         routes
      app.config.ts         app settings and providers
    index.html              main HTML page
    main.ts                 starting point
    styles.css              global styles
  angular.json              project settings
  package.json              dependencies
```

Create a project with `ng new my-app` and run it with `ng serve` (opens on `http://localhost:4200`).

Angular vs React

Point	Angular	React
Type	Full framework	UI library
Language	TypeScript (default)	JavaScript or TypeScript
Built-in tools	Routing, forms, HttpClient	Needs extra libraries
Templates	HTML templates with directives	JSX
Learning curve	Steeper	Easier to start

4. Angular Forms and Services

Services and dependency injection

- A service is a class that holds logic or data shared by many components, such as API calls.
- Dependency injection (DI) means Angular creates the service and gives it to the component. We do not create it ourselves with `new`.

```
// api.service.ts
import { Injectable, inject } from '@angular/core';
import { HttpClient } from '@angular/common/http';

@Injectable({ providedIn: 'root' })
export class ApiService {
  private http = inject(HttpClient);
  private url = 'http://127.0.0.1:5000';

  getHello() {
    return this.http.get<{ message: string }>(`${this.url}/hello`);
  }

  greet(name: string) {
    return this.http.post<{ message: string }>(`${this.url}/greet`, { name });
  }
}
```

Using HttpClient

Add `provideHttpClient()` in `app.config.ts` first. HttpClient methods return Observables, so we use `subscribe()` to get the data.

```
// app.config.ts
import { provideHttpClient } from '@angular/common/http';

export const appConfig = {
  providers: [provideRouter(routes), provideHttpClient()],
};

// in a component
private api = inject(ApiService);

ngOnInit() {
  this.api.getHello().subscribe(res => this.message = res.message);
}
```

Connecting a form to the API (Reactive Form)

```
// greet.component.ts
import { Component, inject } from '@angular/core';
import { FormBuilder, ReactiveFormsModule, Validators } from '@angular/forms';
import { ApiService } from './api.service';

@Component({
  selector: 'app-greet',
  standalone: true,
  imports: [ReactiveFormsModule],
  template: `
    <form [formGroup]="form" (ngSubmit)="submit()">
      <input formControlName="name" placeholder="Your name" />
      <button type="submit" [disabled]="form.invalid">Send</button>
    </form>
    <p>{{ result }}</p>
  `,
})
export class GreetComponent {
  private fb = inject(FormBuilder);
  private api = inject(ApiService);
  result = '';

  form = this.fb.group({ name: ['', Validators.required] });

  submit() {
    this.api.greet(this.form.value.name!).subscribe(res => this.result = res.message);
  }
}
```

Reactive Forms vs Template-driven Forms

Point	Reactive Forms	Template-driven Forms
Where defined	In the TypeScript class	In the HTML template
Directive	formGroup, formControlName	ngModel
Validation	Validators in code	HTML attributes (required, etc.)
Best for	Bigger, complex forms	Small, simple forms
Testing	Easier	Harder

5. Prompt Engineering 101

What is prompt engineering?

Prompt engineering is writing clear instructions for an AI model so that it gives a useful answer. A better prompt usually gives a better result.

OpenAI Playground

The OpenAI Playground is a web page where we can test prompts without writing code. We can:

- Choose a model and write a system message and a user message.
- Change settings such as temperature (lower = more fixed answers, higher = more creative) and max tokens.
- Run the prompt, compare outputs, and view the equivalent API code.

How to write effective prompts

- Be clear and specific about the task.
- Give context (who the answer is for, what the topic is).
- Say the output format you want (list, table, JSON, number of lines).

- Set limits such as length or tone.
- Give an example when the format matters.
- If the result is not good, change the prompt and try again.

Zero-shot vs Few-shot prompting

Type	Meaning	When to use
Zero-shot	Only the instruction. No examples.	Simple, common tasks
Few-shot	Instruction plus a few examples.	When the output format or style must be exact

Prompt examples (illustrative)

```
Weak prompt:
  Tell me about Flask.

Better prompt:
  Explain Flask in 3 simple bullet points for a beginner student.
  Mention what it is, one use, and one benefit.

Zero-shot example:
  Classify the sentiment of this review as Positive or Negative:
  "The app is very slow."

Few-shot example:
  Classify the sentiment as Positive or Negative.
  Review: "Loved the design."   Sentiment: Positive
  Review: "Crashes every time." Sentiment: Negative
  Review: "The app is very slow." Sentiment:
```

> *These prompts are examples to show the idea. Model outputs can differ each time, so no fixed output is claimed here.*

6. Implementation Evidence

> *Status note: the items below are practice tasks with the output expected from the code in this report. Replace the "Result" column with your own observed result (and add screenshots) before submitting.*

Sample API requests and responses (illustrative)

```
GET /hello
Response 200:
{"message": "Hello from Flask"}

POST /greet
Request body:
{"name": "Siddharth"}
Response 200:
{"message": "Hello Siddharth"}
```

Test from the terminal:

```
curl http://127.0.0.1:5000/hello
curl -X POST http://127.0.0.1:5000/greet \
     -H "Content-Type: application/json" \
     -d '{"name": "Siddharth"}'
```

Simple frontend-backend integration example

A plain HTML page that calls the Flask API with `fetch()`:

```
<button onclick="callApi()">Say Hello</button>
<p id="output"></p>

<script>
  function callApi() {
    fetch("http://127.0.0.1:5000/hello")
      .then(res => res.json())
      .then(data => {
        document.getElementById("output").textContent = data.message;
      });
  }
</script>
```

With the Angular version, the same call is made through `ApiService` and `HttpClient` (Section 4), and the Flask app must allow origin `http://localhost:4200` using CORS.

Practical tasks and expected output

Task	Expected output	Result (fill in)
Run Flask app and open /hello	Browser shows {"message": "Hello from Flask"}	
Call /hello from JavaScript fetch()	Page shows "Hello from Flask" after button click	
Call API from another origin without CORS	Browser console shows CORS error	
Add flask-cors and retry	Request succeeds, no CORS error	
Create Angular app and add routes	URLs / and /about show different components	
Create ApiService with HttpClient	Component shows message from Flask	
Reactive form posting to /greet	Typing a name and submitting shows "Hello <name>"	
Try zero-shot and few-shot prompts in Playground	Compare the two outputs and note the difference	